Using the Activepieces React UI SDK in Other Projects

This guide explains how to embed the Activepieces React UI SDK (flow builder, dashboard, connections, runs, templates) into other applications using the npm package. The SDK is loaded via a script tag and exposes React components that you mount into your app.

Overview

- **Package**: `@activepieces/react-ui-sdk` (bundled build from `dist/packages/react-ui-sdk-bundled`)
- **Loading**: Script tag (not ESM import) – the bundle sets `window.__AP_SDK_MODULE__`
- **Styles**: Inlined in the bundle via style-loader – no separate stylesheet required
- **Framework**: Works in any host app (Angular, Vue, plain HTML/JS) that can inject a script and mount React components

Prerequisites

1. **Activepieces backend** running (e.g. `http://localhost:3000`)
2. **Built SDK bundle** in the activepieces repo (see BUILD_GUIDE.md)

Step 1: Build the SDK (Activepieces Repo)

From the activepieces workspace root:

```
cd /path/to/activepieces
npx nx run react-ui-sdk:bundle
npx nx run react-ui-sdk:fix-bundle-package
```

`fix-bundle-package` updates `package.json` so the bundle works outside the monorepo (correct `main`, replaces `workspace:*` for `@activepieces/shared`).

Step 2: Install in Your Project

Option A: Local path (development)

```
{
  "dependencies": {
    "@activepieces/react-ui-sdk": "file:../activepieces/dist/packages/react-ui-sdk-bundled"
  }
}
```

Adjust the path so it correctly points to your activepieces `dist` folder.

Option B: Published package (if you publish to npm)

```
{
  "dependencies": {
    "@activepieces/react-ui-sdk": "^1.0.0"
  }
}
```

Then run:

```
bun install   # or npm install / yarn install
```

Step 3: Copy SDK Files into Your App's Output

The SDK must be served from your app (e.g. under `/sdk/`). Copy the following from `node_modules/@activepieces/react-ui-sdk` into your build output:

Glob	Output	Purpose
`index.js`	`sdk/`	Main SDK bundle (~12 MB)
`*.woff2`	`sdk/`	Font files emitted by webpack
`locales/**/*`	`sdk/`	i18n translation files
`assets/**/*`	`sdk/`	Images (e.g. ada-logo, todo guides)
`ada-logo.png`	`.` (root)	Default platform branding (see below)

Example: Angular `angular.json`

```
"assets": [
  { "glob": "index.js", "input": "node_modules/@activepieces/react-ui-sdk", "output": "sdk" },
  { "glob": "*.woff2", "input": "node_modules/@activepieces/react-ui-sdk", "output": "sdk" },
  { "glob": "locales/**/*", "input": "node_modules/@activepieces/react-ui-sdk", "output": "sdk" },
  { "glob": "assets/**/*", "input": "node_modules/@activepieces/react-ui-sdk", "output": "sdk" },
  { "glob": "ada-logo.png", "input": "node_modules/@activepieces/react-ui-sdk/assets", "output": "." }
]
```

Example: Vite `vite.config.ts`

Use `vite-plugin-static-copy` or similar to copy the SDK folder into `dist/sdk/`.

Example: Plain HTML / Static Site

Manually copy the bundle output to your static assets folder (e.g. `public/sdk/`) so the script and assets are served under `/sdk/`.

Step 4: Configure Before Loading the Script

Before loading `/sdk/index.js`, set these on `window`:

```
// Required: Base URL for SDK assets (fonts, images).
// Must be the FULL script URL (e.g. .../sdk/index.js).
// Webpack strips the last path segment to get publicPath, so this yields .../sdk/
window.__AP_SDK_BASE_PATH__ = window.location.origin + '/sdk/index.js';

// Required: API and auth config
window.__AP_SDK_CONFIG__ = {
  apiUrl: 'http://localhost:4201',   // Your app origin (for proxy) or direct backend URL
  token: 'your-jwt-token',
  projectId: 'your-project-id',       // Optional
  flowId: 'your-flow-id'              // Optional, required for flow-builder
};
```

Notes:

- `apiUrl`: If you proxy `/api` to the backend, use your app's origin (e.g. `''` or `window.location.origin`). Otherwise use the backend URL directly.
- `token`: JWT from your Activepieces instance.
- `projectId`: Required for dashboard, connections, runs, templates.
- `flowId`: Required only for the `flow-builder` component.

How to obtain token and projectId

Source	How to get token	How to get projectId
Email / password sign-in	`POST /api/v1/authentication/sign-in` with `{ "email": "...", "password": "..." }` → response includes `token`	Same response includes `projectId` (the user's default project). May be `null` for Super Admins / Owners.
Sign-up	`POST /api/v1/authentication/sign-up` with `{ "email": "...", "password": "...", "firstName": "...", "lastName": "..." }` → response includes `token`	Same response includes `projectId` (new user's project).
API key	Create in platform settings; value is `sk-xxx`. Use as `token` (or set `Authorization: Bearer sk-xxx` on API calls).	API keys are platform-scoped. Use `GET /api/v1/projects` with that token to list projects and pick one by `id`.
Federated auth (OAuth)	Redirect to `/api/v1/federated-authn/login`; after callback, `POST /api/v1/federated-authn/claim` with `{ "code": "..." }` → response includes `token`	Same response includes `projectId`.
Managed auth	`POST /api/v1/managed-authn/external-token` with `{ "externalAccessToken": "..." }` → response includes `token`	Same response includes `projectId`.

Source	How to get token	How to get projectId
(external token)		

List projects (when you only have a token):

```
# With proxy: /api → backend
curl -H "Authorization: Bearer <token>" "https://your-app/api/v1/projects"
```

Response shape:
`{ "data": [ { "id": "proj_xxx", "displayName": "...", ... }, ... ], "next": null, "previous": null }`. Use `data[0].id` or let the user choose.

Example: sign-in and use token + projectId for SDK

```
const res = await fetch('/api/v1/authentication/sign-in', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email: 'user@example.com', password: '...' }),
});
const { token, projectId } = await res.json();

window.__AP_SDK_CONFIG__ = {
  apiUrl: window.location.origin,
  token,
  projectId,  // may be null for super admins — use GET /v1/projects to pick one
};
```

Step 5: Load the Script and Mount a Component

```
// 1. Load the script
const script = document.createElement('script');
script.src = '/sdk/index.js';
script.async = true;
script.onload = () => {
  const sdk = window.__AP_SDK_MODULE__;
  if (!sdk) {
    console.error('SDK module not found');
    return;
  }

  // 2. Configure API (if not done earlier)
  sdk.configureAPI({
    apiUrl: window.location.origin,
    token: 'your-jwt-token',
    projectId: 'your-project-id'
  });

  // 3. Mount a component
  const container = document.getElementById('sdk-container');
  const component = sdk.Dashboard;  // or FlowBuilder, Connections, Runs, Templates
  const props = {
    apiUrl: window.location.origin,
    token: 'your-jwt-token',
    projectId: 'your-project-id'
  };

  const root = sdk.mountReactComponent(container, component, props);

  // 4. Unmount on cleanup
  // sdk.unmountReactComponent(root);
};
document.head.appendChild(script);
```

Available Components

Component	Key	Props
Dashboard	`Dashboard`	`apiUrl` , `token` , `projectId`
Flow Builder	`FlowBuilder`	`apiUrl` , `token` , `projectId` , `flowId`
Connections	`Connections`	`apiUrl` , `token` , `projectId`
Runs	`Runs`	`apiUrl` , `token` , `projectId` , `flowId` (opt.)
Templates	`Templates`	`apiUrl` , `token` , `projectId`

Step 6: API and WebSocket Proxy

If your frontend and backend run on different origins, proxy `/api` (and WebSockets) to the backend.

Example: Angular `proxy.conf.json`

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true,
    "pathRewrite": { "^/api": "" },
    "ws": true
  }
}
```

- `pathRewrite` : Removes `/api` prefix so `/api/v1/...` becomes `/v1/...` on the backend.
- `ws: true` : Enables WebSocket proxying for socket.io (required for real-time updates).

Then configure `apiUrl` as your app origin so requests go to `/api/...` and get proxied.

Default stylesheet and customizing the UI

How styles are loaded (style-loader)

The SDK bundle uses **style-loader**: CSS is compiled (Tailwind, theme, fonts) at build time and **injected into the document** when the script runs. No separate `<link href="...">` is required. The styles are added as `<style>` tags to the page `<head>` and apply globally.

You **cannot** swap style-loader for another approach from the host app alone; the bundle is already built. To ship a separate stylesheet (e.g. for cache or CSP), the SDK would need a different build (e.g. MiniCssExtractPlugin) in the activepieces repo.

What the default stylesheet includes

The inlined styles come from react-ui's main styles and include:

Category	Details
Tailwind	Tailwind 4 utilities and components
Theme	CSS custom properties for light/dark and components
Fonts	Inter (400, 700, 800) via `@font-face` (woff2 from bundle)
Animations	tw-animate-css, custom keyframes (accordion, fade, etc.)
Components	Buttons, inputs, sidebar, builder canvas, modals, etc.

CSS variables (theme) you can override

The UI respects CSS variables on `:root` and `.dark` . Overriding them in your app (or in a stylesheet that loads after the SDK) will restyle the embedded UI to match your brand.

Light theme (`:root`):

```
:root {
  /* Primary (buttons, links, focus) — HSL values without hsl() */
  --primary: 257 74% 57%;
  --primary-100: 257 75% 85%;
  --primary-300: 257 74% 25%;
  --primary-foreground: 210 20% 98%;

  /* Background and text */
  --background: theme('colors.white');
  --foreground: theme('colors.neutral.950');

  /* Surfaces */
  --muted: theme('colors.neutral.100');
  --muted-foreground: theme('colors.neutral.500');
  --border: theme('colors.neutral.200');
  --input: theme('colors.neutral.200');
  --ring: theme('colors.neutral.400');

  /* Radius */
  --radius: 0.625rem;

  /* Sidebar */
  --sidebar-background: theme('colors.neutral.50');
  --sidebar-foreground: theme('colors.neutral.950');
  --sidebar-primary: 257 74% 57%;
  /* ... and others */
}
```

Dark theme (`.dark`): Same variable names are set under `.dark` for dark mode.

Branding from backend: If the platform has custom branding, the SDK's `ThemeProvider` sets `--primary` , `--primary-100` , and `--primary-300` from the API. Your overrides will still apply if they load after the SDK or have higher specificity.

How to customize styles to match your UI

1. Load your own stylesheet after the SDK

Load a CSS file **after** the SDK script so your rules win when specificity is equal:

```
<script src="/sdk/index.js"></script>
<link rel="stylesheet" href="/your-app/sdk-overrides.css" />
```

Or inject the link once the script has run:

```
const link = document.createElement('link');
link.rel = 'stylesheet';
link.href = '/your-app/sdk-overrides.css';
document.head.appendChild(link);
```

2. Override CSS variables

In that stylesheet (or any global CSS), override the theme variables so the SDK uses your colors and radius:

```
/* sdk-overrides.css — match your app's primary and radius */
:root {
  --primary: 220 90% 56%;
  --primary-100: 220 90% 92%;
  --primary-300: 220 90% 30%;
  --primary-foreground: 0 0% 100%;
  --radius: 0.5rem;
}

.dark {
  --primary: 220 90% 60%;
  --primary-100: 220 90% 25%;
  --primary-300: 220 90% 88%;
}
```

3. Scope overrides to the SDK container

To avoid affecting the rest of your app, wrap the SDK mount point and scope overrides under a class:

```
<div class="ap-sdk-host">
  <div id="sdk-container"></div>
</div>


.ap-sdk-host {
  --primary: 220 90% 56%;
  --radius: 0.5rem;
}

.ap-sdk-host .border {
  border-color: var(--your-app-border);
}
```

4. Override specific components

Use the same class or data attributes the SDK uses, or target by structure. Inspect the DOM to get exact selectors:

```
.ap-sdk-host [data-slot="sidebar"] {
  background: var(--your-sidebar-bg);
}

.ap-sdk-host button[class*="primary"] {
  border-radius: 8px;
}
```

5. Font

The default font is **Inter**. To use your font:

- Override in your stylesheet (load after SDK):

  ```
  .ap-sdk-host {
    font-family: 'Your Font', sans-serif;
  }
  ```

- Or override the variable the SDK uses:

```
:root {
  --font-sans: 'Your Font', ui-sans-serif, system-ui, sans-serif;
}
```

Summary

Goal	Approach
Use default look	Nothing; styles are in the bundle
Match your brand (colors, radius)	Load a CSS file after the SDK and set `--primary`, `--radius`, etc. on `:root` or `.ap-sdk-host`
Change fonts	Override `font-family` or `--font-sans` on the SDK container or `:root`
Tweak specific components	Add scoped rules (e.g. under `.ap-sdk-host`) with higher specificity
Avoid affecting your app	Wrap the SDK in a div and put all overrides under that class

You do **not** need to change style-loader in the host app; customization is done by loading your own CSS and overriding the same variables and classes the default stylesheet uses. Reference below.

Variables and classes reference

CSS variables (set on `:root` and `.dark`)
Use HSL values **without** the `hsl()` wrapper (e.g. `257 74% 57%`). The stylesheet wraps them in `hsl(var(--primary))` etc.

Variable	Purpose
`--background`, `--foreground`	Page background and text
`--primary`, `--primary-100`, `--primary-300`, `--primary-foreground`	Primary brand (buttons, links, focus)
`--secondary`, `--secondary-foreground`	Secondary buttons/surfaces
`--muted`, `--muted-foreground`	Muted backgrounds and secondary text
`--accent`, `--accent-foreground`	Hover/selected accents
`--destructive`, `--destructive-100`, `--destructive-300`	Error/danger actions
`--success`, `--success-100`, `--success-300`	Success states
`--warning`, `--warning-100`, `--warning-300`	Warning states
`--border`, `--input`, `--ring`	Borders, inputs, focus ring
`--radius`	Base border radius (derived: `--radius-lg`, `--radius-md`, `--radius-sm`)
`--sidebar-background`, `--sidebar-foreground`, `--sidebar-primary`, `--sidebar-primary-foreground`, `--sidebar-accent`, `--sidebar-accent-foreground`, `--sidebar-border`, `--sidebar-ring`, `--sidebar-progress`	Sidebar theming

Variable	Purpose
`--popover`	Popover/dropdown background
`--builder-background` , `--builder-background-pattern`	Flow builder canvas
`--add-button-shadow`	“Add step” button focus shadow
`--font-sans`	Default font stack (default: Inter)
`--chart-1` … `--chart-5`	Chart colors
`--light-blue`	Misc UI (e.g. placeholders)

Tailwind/theme utility classes

These map to the variables above. Overriding the variables changes these automatically; you can also override the class output in your CSS.

Class pattern	Maps to
`bg-background` , `text-foreground`	Page
`bg-primary` , `text-primary-foreground` , `bg-primary-100` , `text-primary` , `bg-primary-300`	Primary
`bg-secondary` , `text-secondary-foreground`	Secondary
`bg-muted` , `text-muted-foreground`	Muted
`bg-accent` , `text-accent-foreground`	Accent
`bg-destructive` , `text-destructive-foreground` , `bg-destructive-100` , etc.	Destructive
`bg-success` , `text-success-foreground` , etc.	Success
`border-border` , `border-input` , `ring-ring`	Borders and ring
`rounded-lg` , `rounded-md` , `rounded-sm`	`--radius-lg` , `--radius-md` , `--radius-sm` (from `--radius`)
`font-sans`	`--font-sans`
`bg-sidebar` , `text-sidebar-foreground` , `bg-sidebar-primary` , etc.	Sidebar

Other notable classes / selectors

Use these if you need to target specific UI parts (prefer variables when possible):

Selector / class	Purpose
`.dark`	Wrapper for dark mode (variables under `.dark`)
`.scrollbar-hover` , `.scrollbar-none`	Custom scrollbar utilities
`.react-flow`	Flow builder canvas (React Flow); also uses `--primary-100` , `--primary-300` for selection
`.tiptap`	Rich text editor (notes, descriptions)
`.note-node`	Note nodes in the flow builder

Selector / class	Purpose
`[data-state="open"]` , `[data-state="closed"]`	Radix UI open/closed state (dialogs, dropdowns)
`[data-slot="sidebar"]`	Sidebar slot (if present in your build)

Complete Angular Example

1. `package.json`

```
{
  "dependencies": {
    "@activepieces/react-ui-sdk": "file:../activepieces/dist/packages/react-ui-sdk-bundled",
    "@activepieces/shared": "file:../activepieces/packages/shared"
  }
}
```

2. `angular.json` assets

See Step 3.

3. `proxy.conf.json`

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true,
    "pathRewrite": { "^/api": "" },
    "ws": true
  }
}
```

4. Wrapper Component (`activepieces.component.ts`)

```
@Component({
  selector: 'app-activepieces',
  template: `<div #reactContainer class="react-mount-point"></div>`,
})
export class ActivepiecesComponent implements AfterViewInit, OnDestroy {
  @ViewChild('reactContainer', { static: true }) reactContainer!: ElementRef<HTMLDivElement>;
  @Input() apiUrl = '';
  @Input() token = '';
  @Input() projectId?: string;
  @Input() flowId?: string;
  @Input() component: 'flow-builder' | 'dashboard' | 'connections' | 'runs' | 'templates' = 'dashboard';

  private reactRoot: any = null;
  private sdkModule: any = null;

  async ngAfterViewInit() {
    const effectiveApiUrl = this.apiUrl || window.location.origin;

    // Set base path and config BEFORE loading script
    (window as any).__AP_SDK_BASE_PATH__ = window.location.origin + '/sdk/index.js';
    (window as any).__AP_SDK_CONFIG__ = {
      apiUrl: effectiveApiUrl,
      token: this.token,
      projectId: this.projectId,
      flowId: this.flowId,
    };

    if (this.token) {
      localStorage.setItem('token', this.token);
      if (this.projectId) localStorage.setItem('projectId', this.projectId);
    }

    // Load script
    this.sdkModule = await this.loadSDKModule();
    const componentMap = {
      'flow-builder': 'FlowBuilder',
      'dashboard': 'Dashboard',
      'connections': 'Connections',
      'runs': 'Runs',
      'templates': 'Templates',
    };
    const ReactComponent = this.sdkModule[componentMap[this.component]];
    const props = { apiUrl: effectiveApiUrl, token: this.token, projectId: this.projectId, flowId: this.flowId };
    this.reactRoot = this.sdkModule.mountReactComponent(
      this.reactContainer.nativeElement,
      ReactComponent,
      props
    );
  }

  loadSDKModule(): Promise<any> {
    return new Promise((resolve, reject) => {
      if ((window as any).__AP_SDK_MODULE__) {
        resolve((window as any).__AP_SDK_MODULE__);
        return;
```

```
      }
      const script = document.createElement('script');
      script.src = '/sdk/index.js';
      script.onload = () => {
        const check = () => (window as any).__AP_SDK_MODULE__ && resolve((window as any).__AP_SDK_MODULE__);
        if (check()) return;
        const id = setInterval(() => { if (check()) clearInterval(id); }, 100);
        setTimeout(() => { clearInterval(id); if (!(window as any).__AP_SDK_MODULE__) reject(new Error('SDK not four
      };
      script.onerror = () => reject(new Error('Failed to load SDK'));
      document.head.appendChild(script);
    });
  }

  ngOnDestroy() {
    if (this.reactRoot && this.sdkModule?.unmountReactComponent) {
      this.sdkModule.unmountReactComponent(this.reactRoot);
    }
  }
}
```

5. Template usage

```
<app-activepieces
  apiUrl=""
  token="your-jwt-token"
  projectId="your-project-id"
  component="dashboard">
</app-activepieces>
```

Why `ada-logo.png` at Root?

The default platform theme returns `fullLogoUrl: '/ada-logo.png'` and `logoIconUrl: '/ada-logo.png'`. The UI uses these as `img src`, so the browser requests `http://your-app/ada-logo.png`. Copying `ada-logo.png` to the app root avoids a 404. You can instead change platform branding to use a different URL.

Troubleshooting

Issue	Cause	Fix
Fonts/images 404	Wrong `publicPath`	Set `__AP_SDK_BASE_PATH__` to full script URL (e.g. `origin + '/sdk/index.js'`) **before** loading the script
`/ada-logo.png` 404	Default theme expects logo at root	Copy `ada-logo.png` to app root or update branding
WebSocket fails	Proxy doesn't forward WS	Add `"ws": true` to proxy config and restart dev server

Issue	Cause	Fix
`@activepieces/shared workspace:*` error	Bundle not fixed for external use	Run `npx nx run react-ui-sdk:fix-bundle-package` after bundle
SDK module not found	Script loaded before config	Set `__AP_SDK_CONFIG__` and `__AP_SDK_BASE_PATH__` before injecting the script

Reference: SDK API

After loading the script, `window.__AP_SDK_MODULE__` contains:

```
{
  // Components
  Dashboard: React.ComponentType<DashboardProps>;
  FlowBuilder: React.ComponentType<FlowBuilderProps>;
  Connections: React.ComponentType<ConnectionsProps>;
  Runs: React.ComponentType<RunsProps>;
  Templates: React.ComponentType<TemplatesProps>;

  // Mount helpers
  mountReactComponent(container: HTMLElement, Component: React.ComponentType, props: object): Root;
  unmountReactComponent(root: Root): void;

  // Config
  configureAPI(config: { apiUrl: string; token: string; projectId?: string; flowId?: string }): void;

  // Utilities (optional)
  getAPIUrl(): string | undefined;
  getAuthToken(): string | undefined;
}
```

Related Docs

- BUILD_GUIDE.md – How to build the SDK bundle
- README.md – SDK overview and npm install instructions